

CSS — The Complete Learning Book

From Absolute Beginner to Modern, Production-Ready CSS

A practical course book for learning, teaching, practicing, and building real responsive interfaces.

Includes concepts, syntax, runnable examples, visual diagrams, common mistakes, exercises, mini-projects, and reference sheets.

Table of Contents

1. CSS Fundamentals
2. Selectors
3. Cascade, Specificity and Inheritance
4. Colors
5. Typography and Fonts
6. Backgrounds and Gradients
7. The CSS Box Model
8. CSS Units and Sizing
9. Display and Normal Flow
10. Overflow and Scrolling
11. Positioning and Stacking
12. Flexbox
13. CSS Grid
14. Pseudo-classes and Pseudo-elements
15. Responsive Design and Media Queries
16. Images, Aspect Ratio and Object Fitting
17. Borders, Radius, Shadows and Visual Effects
18. Transitions and Transforms
19. CSS Animations
20. CSS Variables and Reusable Design Tokens

21. CSS Functions: calc, min, max and clamp
22. Forms and UI Controls
23. Lists and Tables
24. Modern CSS Features
25. Accessibility with CSS
26. DevTools and CSS Debugging
27. CSS Architecture and Best Practices
28. CSS Performance
29. Project-Based Learning
30. Final Capstone
31. CSS Quick Reference

Chapter 1: CSS Fundamentals

CSS (Cascading Style Sheets) controls the presentation and layout of HTML. HTML gives a document structure; CSS decides how that structure looks and behaves across screen sizes.

What CSS does

- Styles text, colors, backgrounds, borders and shadows.
- Controls spacing, sizing and layout.
- Creates responsive interfaces for phones, tablets and desktops.
- Adds transitions and animations.
- Supports modern layout systems such as Flexbox and Grid.

Three ways to add CSS

```
<!-- Inline -->
<p style="color: red;">Hello</p>

<!-- Internal -->
<style>
  p { color: red; }
</style>

<!-- External -->
<link rel="stylesheet" href="style.css">
```

Recommended: Use external stylesheets for real projects. Keep styles separate from HTML so they are easier to maintain.

CSS syntax

```
selector {  
  property: value;  
}
```

```
SELECTOR → property → value  
p        {   color    : red;   }
```

Comments

```
/* This is a CSS comment */
```

Practice

1. Create an HTML page with a heading, paragraph and button.
2. Style the three elements using an external stylesheet.
3. Change the text color, background, spacing and font size.

Chapter 2: Selectors

Selectors tell CSS which elements should receive a rule.

Basic selectors

```
* { box-sizing: border-box; } /* universal */  
  
p { color: gray; }           /* element */  
  
.card { padding: 20px; }     /* class */  
  
#hero { min-height: 400px; } /* ID */  
  
h1, h2, h3 { line-height: 1.2; } /* grouping */
```

Combinators

Selector	Meaning	Example
A B	Descendant	.card p
A > B	Direct child	.card > p
A + B	Immediately following sibling	h2 + p
A ~ B	Following sibling	h2 ~ p

```
nav a { text-decoration: none; }  
.card > img { width: 100%; }  
h2 + p { margin-top: 0; }
```

Attribute selectors

```
input[type="email"] { ... }  
a[target="_blank"] { ... }  
button[disabled] { ... }
```

Practice

4. Build a navigation bar and style only links inside the navigation.
5. Create three buttons with different data attributes and select them.

6. Use child and sibling combinators to style a small article.

Chapter 3: Cascade, Specificity and Inheritance

The cascade determines which declaration wins when multiple CSS rules target the same property.

Specificity

Selector	Relative strength
Element: p	Low
Class / attribute / pseudo-class: .card, [type], :hover	Medium
ID: #hero	High
Inline style	Very high

```
p { color: blue; }
.text { color: green; }
#intro { color: purple; }
```

Rule of thumb: Prefer low, predictable specificity. Avoid using IDs for ordinary component styling.

Inheritance

Properties such as color and font-family often inherit from parents. Layout properties such as margin and padding do not normally inherit.

```
body {
  font-family: Arial, sans-serif;
  color: #222;
}

button {
  font: inherit;
}
```

!important

```
/* Use rarely */
color: red !important;
```

Best practice: Fix specificity or architecture instead of reaching for !important.

Practice

7. Create competing selectors and predict which rule wins before checking the browser.
8. Refactor a stylesheet that uses many IDs and !important declarations.

Chapter 4: Colors

CSS supports named colors, hexadecimal, RGB, HSL and modern color functions.

```
color: red;
color: #2563eb;
color: rgb(37 99 235);
color: hsl(221 83% 53%);
color: rgb(37 99 235 / 50%);
```

Text and background colors

```
body {
  color: #222;
  background: #f5f5f5;
}

.card {
  background-color: white;
}
```

Transparency

```
background: rgb(255 255 255 / 0.15);
```

Teaching tip: Explain contrast, not only color syntax. A good interface needs readable text and visible controls.

Practice

9. Create a small theme with primary, secondary, surface and text colors.
10. Create a button with a hover state using a different color.

Chapter 5: Typography and Fonts

Typography controls the visual structure and readability of text.

Property	Purpose
font-family	Typeface stack
font-size	Text size
font-weight	Thickness
font-style	Italic/normal
line-height	Vertical rhythm
letter-spacing	Character spacing
word-spacing	Word spacing
text-align	Horizontal text alignment
text-decoration	Underline/line-through
text-transform	Uppercase/lowercase/capitalization

```
body {
  font-family: Arial, sans-serif;
  font-size: 16px;
  line-height: 1.5;
}

h1 {
  font-size: 2.5rem;
  font-weight: 700;
  letter-spacing: -0.02em;
}
```

Text overflow

```
.title {  
  white-space: nowrap;  
  overflow: hidden;  
  text-overflow: ellipsis;  
}
```

Practice

11. Create a typographic scale for h1, h2, h3, body and small text.
12. Make a long product title truncate with an ellipsis.

Chapter 6: Backgrounds and Gradients

Background properties control images, positioning, sizing, repetition and gradients.

```
.hero {  
  background-image: url("hero.jpg");  
  background-size: cover;  
  background-position: center;  
  background-repeat: no-repeat;  
}
```

Linear gradient

```
.banner {  
  background: linear-gradient(135deg, #111, #444);  
}
```

Radial gradient

```
.spot {  
  background: radial-gradient(circle, #fff, #ddd);  
}
```

Multiple backgrounds

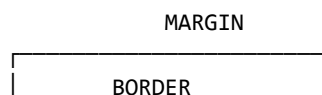
```
.card {  
  background:  
    linear-gradient(rgba(0 0 0 / .3), rgba(0 0 0 / .3)),  
    url("photo.jpg") center / cover;  
}
```

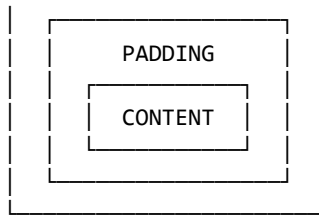
Practice

13. Build a hero section with a background image and readable text.
14. Create a card with a gradient overlay.
15. Experiment with background-position and background-size.

Chapter 7: The CSS Box Model

Every normal element is treated as a box consisting of content, padding, border and margin.





```
.card {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
  margin: 20px;
}
```

box-sizing

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Why: With border-box, the declared width includes content, padding and border. This makes sizing much easier to reason about.

Min and max sizing

```
.container {
  width: 100%;
  max-width: 1200px;
  margin-inline: auto;
}
```

Practice

16. Draw the box model by hand for a 300px box with padding and border.
17. Compare content-box and border-box in DevTools.
18. Build a centered content container with a maximum width.

Chapter 8: CSS Units and Sizing

Use absolute and relative units according to the problem.

Unit	Typical use
px	Borders, precise small dimensions
%	Relative to a containing dimension
rem	Scalable typography and spacing
em	Size relative to current font context
vw/vh	Viewport-relative sizing
dvh	Dynamic viewport height
ch	Text measure
fr	Grid fractions

```
.page {
  width: min(100% - 2rem, 1200px);
  min-height: 100dvh;
}
```

Responsive sizing with clamp

```
h1 {  
  font-size: clamp(2rem, 5vw, 4rem);  
}
```

Practice

19. Create a responsive heading using clamp().
20. Create a page wrapper that never touches the viewport edges.

Chapter 9: Display and Normal Flow

The display property determines how an element participates in layout.

Value	Main idea
block	Starts on a new line and normally fills available inline space
inline	Flows within text; width/height behave differently
inline-block	Flows inline but accepts box dimensions
flex	One-dimensional layout
grid	Two-dimensional layout
none	Removed from layout

```
.hidden {  
  display: none;  
}  
  
.badge {  
  display: inline-block;  
}
```

Visibility

```
.invisible {  
  visibility: hidden;  
}
```

Difference: display:none removes the element from layout. visibility:hidden keeps its layout space while hiding it.

Practice

21. Create examples of block, inline and inline-block elements.
22. Hide an element with both display:none and visibility:hidden and compare.

Chapter 10: Overflow and Scrolling

Overflow controls what happens when content is larger than its box.

```

.panel {
  width: 300px;
  height: 200px;
  overflow: auto;
}

.no-scroll {
  overflow: hidden;
}

.table-wrapper {
  overflow-x: auto;
}

```

Scroll snapping

```

.carousel {
  display: flex;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
}

.slide {
  scroll-snap-align: start;
}

```

Practice

23. Build a scrollable code panel.
24. Create a horizontally scrollable mobile card list.
25. Experiment with scroll snapping.

Chapter 11: Positioning and Stacking

Positioning is used when an element needs a controlled offset or a relationship to another box.

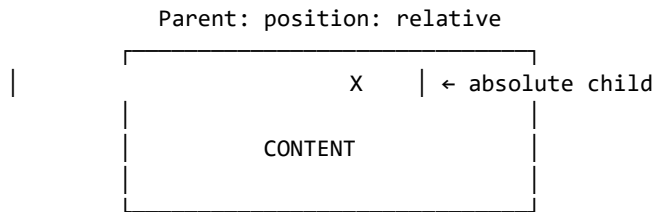
Value	Behavior
static	Default normal flow
relative	Remains in flow; can be offset; establishes a containing block
absolute	Removed from normal flow; positioned against a containing block
fixed	Usually attached to the viewport
sticky	Acts normal until a scroll threshold, then sticks within its context

```

.card {
  position: relative;
}

.badge {
  position: absolute;
  top: 12px;
  right: 12px;
}

```



```
.chat {
  position: fixed;
  right: 1rem;
  bottom: 1rem;
}

.sidebar {
  position: sticky;
  top: 1rem;
}
```

Offsets

```
top: 10px;
right: 20px;
bottom: 10px;
left: 20px;

/* shorthand */
inset: 10px 20px;
```

z-index

```
.modal {
  position: fixed;
  z-index: 1000;
}
```

Common mistake: Do not use absolute positioning to build an entire page. Use Flexbox or Grid for primary layout.

Practice

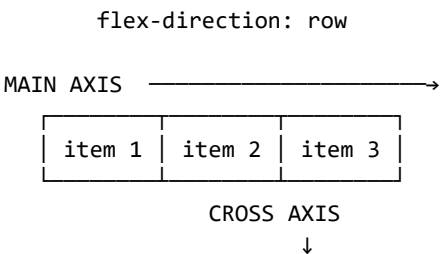
26. Build a product card with an absolute badge.
27. Create a fixed floating action button.
28. Create a sticky sidebar next to long content.

Chapter 12: Flexbox

Flexbox is a one-dimensional layout system. It is ideal for rows, columns, navigation bars, toolbars and component internals.

```
.nav {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}
```

Main and cross axes



Property	Purpose
flex-direction	row, row-reverse, column, column-reverse
justify-content	Distribution along main axis
align-items	Alignment along cross axis
align-content	Distribution of wrapped lines
flex-wrap	Allow items to wrap
gap	Space between items
flex-grow	How much an item can grow
flex-shrink	How much an item can shrink
flex-basis	Initial main size
align-self	Override cross-axis alignment
order	Change visual order

```
.layout {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.item {
  flex: 1 1 240px;
}
```

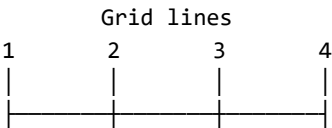
Practice

- 29. Build a responsive navbar with Flexbox.
- 30. Create three cards that wrap on smaller screens.
- 31. Create a centered modal using Flexbox.
- 32. Use flex-grow and flex-basis to create unequal columns.

Chapter 13: CSS Grid

Grid is a two-dimensional layout system based on rows and columns.

```
.products {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```



1	2	3
4	5	6

```
.featured {
  grid-column: 1 / 3;
}

.tall {
  grid-row: span 2;
}
```

Responsive Grid

```
.products {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

Grid areas

```
.page {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 220px 1fr;
}

header { grid-area: header; }
aside { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }
```

Practice

33. Build a three-column card grid.
34. Make one card span two columns.
35. Build a dashboard using named grid areas.
36. Rebuild the same layout with Flexbox and compare the two systems.

Chapter 14: Pseudo-classes and Pseudo-elements

Pseudo-classes select elements based on state or position; pseudo-elements target generated or special portions of an element.

```
.button:hover { transform: translateY(-2px); }
input:focus { outline: 2px solid currentColor; }
input:focus-visible { outline: 2px solid currentColor; }
li:nth-child(2) { font-weight: 700; }
button:disabled { opacity: .5; }
.card:not(.featured) { border: 1px solid #ddd; }
```

Modern selectors

```
.form:has(input:invalid) { ... }

:is(h1, h2, h3) {
  line-height: 1.2;
}

p:where(.muted, .secondary) {
  color: #666;
}
```

Pseudo-elements

```
.link::after {
  content: " ↗";
}

.quote::before {
  content: "“";
}
```

Practice

- 37. Create hover, focus and disabled states for a button.
- 38. Create a custom decorative underline with ::after.
- 39. Use :nth-child() to style alternating list items.
- 40. Try :has() on a form container.

Chapter 15: Responsive Design and Media Queries

Responsive design lets one interface adapt to different screen sizes and input conditions.

Mobile-first

```
.cards {
  display: grid;
  grid-template-columns: 1fr;
}

@media (min-width: 768px) {
  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1100px) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Useful media queries

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    scroll-behavior: auto !important;
  }
}

@media (hover: hover) {
  .card:hover {
    transform: translateY(-2px);
  }
}
```

Container queries

```
.card-grid {
  container-type: inline-size;
}

@container (min-width: 500px) {
  .card {
    grid-template-columns: 160px 1fr;
  }
}
```

Practice

41. Convert a desktop-first page into mobile-first CSS.
42. Build a responsive navbar that changes layout at a chosen breakpoint.
43. Use container queries so a card adapts to its container rather than the viewport.

Chapter 16: Images, Aspect Ratio and Object Fitting

Images often need controlled dimensions while preserving their intrinsic proportions.

```
.avatar {
  width: 80px;
  aspect-ratio: 1;
  border-radius: 50%;
  object-fit: cover;
}

.product-image {
  width: 100%;
  aspect-ratio: 4 / 3;
  object-fit: cover;
  object-position: center;
}
```

Property	Purpose
object-fit: cover	Fill box, crop overflow
object-fit: contain	Fit whole image, preserve full image
object-position	Control crop/focal point
aspect-ratio	Set a preferred width-to-height ratio

Practice

- 44. Create a responsive product image with a fixed aspect ratio.
- 45. Create an avatar that stays circular.
- 46. Compare cover and contain.

Chapter 17: Borders, Radius, Shadows and Visual Effects

These properties give components depth and shape.

```
.card {  
  border: 1px solid #ddd;  
  border-radius: 16px;  
  box-shadow: 0 10px 30px rgb(0 0 0 / .08);  
}  
  
button {  
  outline: 2px solid transparent;  
  outline-offset: 3px;  
}
```

Blur and glass effects

```
.glass {  
  background: rgb(255 255 255 / .15);  
  backdrop-filter: blur(12px);  
}
```

Filters

```
.muted-image {  
  filter: grayscale(1);  
}  
  
.soft {  
  filter: blur(2px);  
}
```

Caution: Filters and backdrop-filter can be visually expensive. Use them where they add clear value.

Practice

- 47. Create three card styles using border, radius and shadow.
- 48. Build a simple glass-style panel over a background image.

Chapter 18: Transitions and Transforms

Transitions animate a change between two states. Transforms change an element's visual geometry without changing normal document flow.

```

.card {
  transition:
    transform 180ms ease,
    box-shadow 180ms ease;
}

.card:hover {
  transform: translateY(-4px) scale(1.01);
}

```

Transform	Effect
translate()	Move
scale()	Resize visually
rotate()	Rotate
skew()	Skew
transform-origin	Change transformation anchor

Practice

49. Build a button with a subtle hover transition.
50. Create a card lift effect.
51. Create a rotating icon and control its transform origin.

Chapter 19: CSS Animations

Keyframe animations define a sequence of visual states.

```

@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.05); }
  100% { transform: scale(1); }
}

.loader {
  animation:
    pulse 1.2s ease-in-out infinite;
}

```

Property	Purpose
animation-name	Keyframe name
animation-duration	Length of one cycle
animation-delay	Wait before starting
animation-iteration-count	Number of repetitions
animation-direction	Normal/reverse/alternate
animation-fill-mode	Styles before/after animation
animation-play-state	Running/paused

Practice

52. Build a loading indicator.
53. Build a slide-in notification.
54. Add reduced-motion behavior.

Chapter 20: CSS Variables and Reusable Design Tokens

Custom properties make repeated values easy to change and enable themes.

```
:root {
  --color-primary: #2563eb;
  --color-text: #222;
  --space-1: .25rem;
  --space-2: .5rem;
  --radius-md: 12px;
}

.button {
  background: var(--color-primary);
  color: white;
  padding: var(--space-2) var(--space-1);
  border-radius: var(--radius-md);
}
```

Fallback

```
color: var(--brand-color, #2563eb);
```

Theme concept

```
:root {
  --surface: white;
  --text: #111;
}

[data-theme="dark"] {
  --surface: #111;
  --text: #eee;
}

body {
  background: var(--surface);
  color: var(--text);
}
```

Practice

55. Create a token system for spacing, colors, radius and shadows.
56. Build a light/dark theme with custom properties.

Chapter 21: CSS Functions: calc(), min(), max() and clamp()

CSS functions allow values to respond to available space and constraints.

```

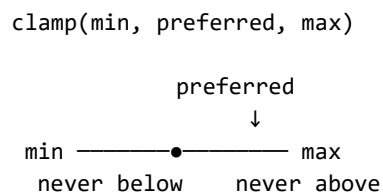
.content {
  width: calc(100% - 2rem);
  max-width: 1200px;
}

.wrapper {
  width: min(100% - 2rem, 1200px);
}

.panel {
  width: max(300px, 40%);
}

h1 {
  font-size: clamp(2rem, 5vw, 4rem);
}

```



Practice

57. Build a fluid heading scale with clamp().
58. Use min() to create a responsive page wrapper.
59. Use calc() to account for fixed side padding.

Chapter 22: Forms and UI Controls

Forms need clear states, usable sizing, and strong focus indicators.

```

input,
textarea,
select,
button {
  font: inherit;
}

input {
  width: 100%;
  padding: .75rem 1rem;
  border: 1px solid #bbb;
  border-radius: 8px;
}

input:focus-visible {
  outline: 3px solid currentColor;
  outline-offset: 2px;
}

```

```
input:invalid {  
  border-color: #b91c1c;  
}  
  
button:disabled {  
  cursor: not-allowed;  
  opacity: .6;  
}
```

Accessibility: Never remove focus outlines without replacing them with a clearly visible alternative.

Practice

60. Build a login form.
61. Add normal, focus, invalid, disabled and checked states.
62. Make the form responsive.

Chapter 23: Lists and Tables

Lists and tables are common content structures that benefit from predictable spacing and responsive behavior.

```
ul {  
  list-style: none;  
  padding: 0;  
}  
  
li {  
  padding-block: .5rem;  
}  
  
table {  
  width: 100%;  
  border-collapse: collapse;  
}  
  
th, td {  
  padding: .75rem;  
  text-align: left;  
  border-bottom: 1px solid #ddd;  
}  
  
.table-wrapper {  
  overflow-x: auto;  
}
```

Practice

63. Create a styled feature list.
64. Build a responsive pricing comparison table.

Chapter 24: Modern CSS Features

Modern CSS reduces the need for JavaScript or complicated selector workarounds.

Nesting

```
.card {  
  padding: 1rem;  
  
  h2 {  
    margin: 0;  
  }  
  
  &:hover {  
    transform: translateY(-2px);  
  }  
}
```

Cascade layers

```
@layer reset, base, components, utilities;  
  
@layer reset {  
  * { box-sizing: border-box; }  
}  
  
@layer components {  
  .button { ... }  
}
```

Logical properties

```
.box {  
  margin-inline: auto;  
  padding-block: 1rem;  
  padding-inline: 1.5rem;  
  border-inline-start: 4px solid currentColor;  
}
```

Why logical properties: They adapt better to different writing directions and internationalized interfaces.

Practice

65. Rewrite a component using native CSS nesting.
66. Organize a small stylesheet with cascade layers.
67. Replace left/right spacing with logical properties.

Chapter 25: Accessibility with CSS

Accessible CSS is about more than visual appearance. It must support keyboard users, readable content, different motion preferences and varied devices.

- Maintain visible keyboard focus.
- Do not rely on color alone to communicate meaning.
- Use readable font sizes and line heights.
- Avoid excessive animation and respect prefers-reduced-motion.
- Ensure controls have sufficient target size and spacing.
- Allow text and containers to grow without breaking the layout.

```

:focus-visible {
  outline: 3px solid currentColor;
  outline-offset: 3px;
}

@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}

```

Practice

68. Audit one of your projects for keyboard focus.
69. Test a layout with browser zoom and larger text.
70. Add reduced-motion support.

Chapter 26: DevTools and CSS Debugging

Professional CSS development requires learning how to inspect the browser's computed layout.

Debugging checklist

71. Inspect the element.
72. Check whether the selector matches.
73. Check crossed-out declarations.
74. Open Computed styles and identify the final value.
75. Inspect the box model.
76. Check parent dimensions and overflow.
77. For Flexbox/Grid, enable the browser's layout overlay.
78. Check media-query conditions.
79. Check stacking contexts when z-index behaves unexpectedly.

```

/* Temporary debugging */
* {
  outline: 1px solid rgb(255 0 0 / .15);
}

```

Teaching habit: When a property appears not to work, first ask: Is the selector matching? Is another rule winning? Is the layout context preventing the expected behavior?

Practice

80. Intentionally break a Flexbox layout and debug it with DevTools.
81. Create a specificity conflict and identify the winning declaration.
82. Debug an absolute element that is positioned relative to the wrong ancestor.

Chapter 27: CSS Architecture and Best Practices

As projects grow, organization becomes as important as knowing individual properties.

Component-oriented naming

```
.card {}  
.card__title {}  
.card__body {}  
.card--featured {}
```

Useful principles

- Prefer classes for reusable component styling.
- Keep specificity low.
- Use variables for repeated design decisions.
- Use Flexbox/Grid for layout and absolute positioning for local overlays.
- Keep responsive rules close to the component logic where practical.
- Avoid deeply nested selectors.
- Use meaningful names based on purpose, not appearance.

Suggested stylesheet structure

```
styles/  
  reset.css  
  tokens.css  
  base.css  
  components.css  
  utilities.css  
  responsive.css
```

Practice

83. Refactor one messy stylesheet into logical sections.
84. Replace repeated color and spacing values with variables.
85. Rename classes to describe components rather than colors.

Chapter 28: CSS Performance

Good CSS should be maintainable and should avoid unnecessary work in the browser.

- Avoid huge, overly specific selector chains.
- Avoid excessive animation of expensive properties.
- Prefer transform and opacity for many animations.
- Use images at sensible dimensions and formats.
- Avoid unnecessary filters and backdrop effects.
- Remove unused CSS in production builds.
- Keep component styles predictable.

```
.good-animation {  
  transition: transform 180ms ease, opacity 180ms ease;  
}  
  
.good-animation:hover {  
  transform: translateY(-4px);  
  opacity: .95;  
}
```

Practice

86. Audit a project for unused or duplicated CSS.

87. Replace a layout animation that changes width with a transform where appropriate.

Chapter 29: Project-Based Learning

Projects convert isolated properties into transferable layout skills.

Project 1 — Profile Card

Practice box model, typography, border-radius, shadow, buttons and Flexbox.

Requirements

- Use semantic HTML.
- Use external CSS.
- Use responsive layout.
- Include keyboard focus states.
- Avoid unnecessary absolute positioning.
- Test at narrow and wide widths.

Project 2 — Login Form

Practice forms, focus states, spacing, responsive sizing and accessibility.

Requirements

- Use semantic HTML.
- Use external CSS.
- Use responsive layout.
- Include keyboard focus states.
- Avoid unnecessary absolute positioning.
- Test at narrow and wide widths.

Project 3 — Responsive Navbar

Practice Flexbox, pseudo-classes, mobile-first media queries and sticky positioning.

Requirements

- Use semantic HTML.
- Use external CSS.
- Use responsive layout.
- Include keyboard focus states.
- Avoid unnecessary absolute positioning.

- Test at narrow and wide widths.

Project 4 — Product Grid

Practice Grid, minmax(), auto-fit, object-fit and cards.

Requirements

- Use semantic HTML.
- Use external CSS.
- Use responsive layout.
- Include keyboard focus states.
- Avoid unnecessary absolute positioning.
- Test at narrow and wide widths.

Project 5 — Pricing Section

Practice Grid/Flexbox, typography, buttons, featured cards and responsive behavior.

Requirements

- Use semantic HTML.
- Use external CSS.
- Use responsive layout.
- Include keyboard focus states.
- Avoid unnecessary absolute positioning.
- Test at narrow and wide widths.

Project 6 — Dashboard

Practice Grid areas, sticky sidebar, cards, tables and responsive layout.

Requirements

- Use semantic HTML.
- Use external CSS.
- Use responsive layout.
- Include keyboard focus states.
- Avoid unnecessary absolute positioning.
- Test at narrow and wide widths.

Project 7 — Portfolio Landing Page

Combine typography, backgrounds, Grid, Flexbox, animations and responsive design.

Requirements

- Use semantic HTML.
- Use external CSS.
- Use responsive layout.
- Include keyboard focus states.
- Avoid unnecessary absolute positioning.
- Test at narrow and wide widths.

Chapter 30: Final Capstone

Build a complete responsive website that demonstrates the full CSS skill set.

Capstone brief — Modern Product Website

- Responsive header/navigation.
- Hero section with background image and overlay.
- Product/card grid using auto-fit and minmax().
- Featured product using Grid spanning.
- Responsive pricing section.
- Accessible contact form.
- Sticky secondary navigation or sidebar.
- Subtle transitions and one meaningful animation.
- Light/dark theme using CSS variables.
- Mobile-first media queries and at least one container query.
- Reduced-motion support.
- Responsive table or comparison section.

Suggested folder structure

```
project/  
  index.html  
  css/  
    reset.css  
    tokens.css  
    style.css  
  images/  
    hero.jpg  
    product-1.jpg  
    product-2.jpg
```

Final checklist

Area	Check
HTML	Semantic structure and labels
Selectors	Low and predictable specificity
Box model	box-sizing understood
Flexbox	Correct axis and alignment
Grid	Correct tracks and spans
Responsive	Mobile-first behavior
Forms	Focus and validation states
Motion	Transitions/animations are purposeful
Accessibility	Keyboard, contrast, reduced motion
Architecture	Reusable variables and components

Practice

88. Plan the layout before writing CSS.
89. Build mobile-first.
90. Add desktop enhancements.
91. Test with DevTools.
92. Refactor duplicated CSS.

93. Perform an accessibility and responsive review.

Chapter 31: CSS Quick Reference

Use this section as a compact revision guide after learning the concepts.

Core layout properties

Property	Remember
display	block, inline, inline-block, flex, grid, none
position	static, relative, absolute, fixed, sticky
gap	Space between Flex/Grid items
overflow	Controls overflowing content
z-index	Stacking order within applicable stacking contexts

Flexbox

```
display: flex;
flex-direction: row;
justify-content: center;
align-items: center;
flex-wrap: wrap;
gap: 1rem;
flex: 1 1 240px;
```

Grid

```
display: grid;
grid-template-columns: repeat(3, 1fr);
grid-template-rows: auto;
gap: 1rem;
grid-column: 1 / 3;
grid-row: span 2;
grid-template-areas:
  "header header"
  "sidebar main";
```

Responsive

```
@media (min-width: 768px) {
  /* larger-screen styles */
}

grid-template-columns:
  repeat(auto-fit, minmax(220px, 1fr));

font-size: clamp(1.5rem, 4vw, 3rem);
```

Pseudo-classes

- :hover
- :focus
- :focus-visible
- :active
- :checked
- :disabled
- :nth-child()
- :not()
- :is()
- :where()
- :has()

Pseudo-elements

- ::before
- ::after
- ::first-letter
- ::first-line
- ::selection
- ::placeholder

Modern sizing functions

- calc()
- min()
- max()
- clamp()
- minmax()

Essential properties to master

Category	Properties
Box	width, height, padding, margin, border, box-sizing
Text	font-family, font-size, font-weight, line-height, letter-spacing
Background	background, background-image, background-size, background-position
Layout	display, gap, position, overflow
Flex	flex-direction, justify-content, align-items, flex, flex-wrap
Grid	grid-template-columns, grid-template-rows, grid-column, grid-row, grid-area
Visual	border-radius, box-shadow, opacity, filter
Motion	transition, transform, animation, @keyframes
Modern	var, calc, min, max, clamp, :has, @container, @layer

Final Learning Path

A strong order for teaching or learning CSS is:

94. Fundamentals and selectors
95. Cascade, specificity and inheritance

96. Colors, typography and backgrounds
97. Box model and units
98. Display and normal flow
99. Overflow
100. Positioning
101. Flexbox
102. Grid
103. Pseudo-classes and pseudo-elements
104. Responsive design
105. Images and object fitting
106. Transitions and transforms
107. Animations
108. Variables and CSS functions
109. Forms
110. Modern CSS
111. Accessibility
112. DevTools and debugging
113. Architecture and performance
114. Projects and capstone

End of book — keep practicing by rebuilding real interfaces from screenshots, then refactoring your CSS for responsiveness, accessibility and maintainability.